



**FACULTAD DE
INGENIERÍA**

UNIVERSIDAD DA VINCI
DE GUATEMALA

PROGRAMACIÓN WEB

NOMBRE: JORGE ANDRÉS GALICIA HERRARTE

CARNÉ: 202404125

Parcial Final

Definición del Problema y Solución

El problema:

La industria de los videojuegos ha crecido tanto que los jugadores ahora enfrentan un problema común:

- Hay demasiadas opciones.
- Es difícil encontrar juegos que coincidan con el **estado emocional** del jugador.
- Las plataformas tradicionales no ofrecen recomendaciones basadas en **intereses educativos** o de aprendizaje (ej.: historia, estrategia, creatividad).
- Muchos usuarios pasan más tiempo buscando que jugando.

Esto genera frustración y pérdida de tiempo, especialmente en jóvenes, estudiantes y jugadores casuales.

La Solución: GameMind

GameMind es una aplicación web que recomienda videojuegos basados en:

- Estado de ánimo del usuario (relajado, estresado, triste, curioso, motivado).
- Intereses específicos del usuario (historia, estrategia, educativo, relajante, competitivo, cooperativo).

El sistema muestra recomendaciones personalizadas y permite al usuario calificar si la sugerencia le fue útil.

Justificación de Innovación:

GameMind es innovador porque:

- Combina **emociones + educación + entretenimiento**.
- Introduce el concepto de *gaming emocionalmente inteligente*.
- Permite descubrir juegos que enseñan historia, pensamiento crítico o estrategia.
- En una versión futura puede integrarse con APIs reales (Steam, IGDB, Epic).

Para el prototipo v0, el sistema funciona completamente con datos simulados (*mock API*) para demostrar la idea.

Propuesta Técnica:

Frontend:

Tecnología	Ventajas	Desventajas
React	Muy popular, rápido con Vite, modular, compatible con mock APIs	Requiere estructuración manual
Vue	Fácil curva de aprendizaje	Ecosistema menos amplio
Angular	Incluye todo (router, DI, etc.)	Muy pesado para un MVP

Mi elección final fue: React + Vite

Porque:

- Ideal para MVPs rápidos.
- Excelente rendimiento gracias a Vite.
- Permite integrar fácilmente una “API simulada” sin backend real.
- Facilita migrar a un backend real en el futuro.

Backend:

Arquitectura Planificada para futuro v1

El sistema final incluiría:

- Backend en Node.js + Express
- Base de datos PostgreSQL
- Endpoints REST reales
- Despliegue en Render, Railway o AWS

Arquitectura usada en el Prototipo v0

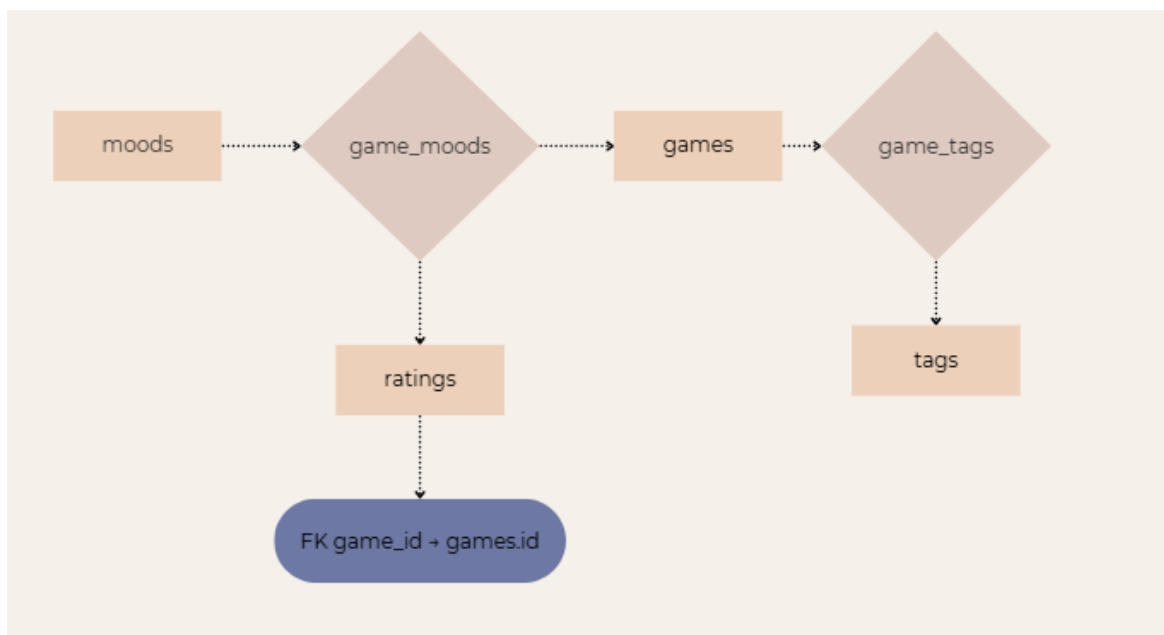
Para este examen, No implemente backend real, sino una API simulada dentro del frontend:

src/services/api.js → fetchMetadata(), getRecommendations(), saveRating()

Esto cumple con:

- Mostrar la funcionalidad completa
- Mantener bajo el nivel de complejidad
- Seguir la rúbrica (API documentada aunque sea simulada)

Persistencia de Datos (Diseño futuro con SQL):



Diseño de la API (Simulada)

Aunque en el prototipo no hay backend real, la API está **documentada como si existiera**, porque así exige la rúbrica.

Todas las respuestas provienen del archivo:

`src/services/api.js`

Endpoint 1: Obtener metadata (modos + tags):

```
{
  "moods": ["relajado", "estresado", "triste", "motivado", "curioso"],
  "tags": ["historia", "estrategia", "relajante", "competitivo", "cooperativo", "educativo"]
}
```

Endpoint 2: Obtener recomendaciones

```
{
  "results": [
    {
      "id": 1,
      "name": "Assassin's Creed Origins",
      "description": "Aventura en Egipto...",
      "main_platform": "PC / Consola"
    }
  ]
}
```

Endpoint 3: Guardar valoración

Body:



Respuesta simulada:



Arquitectura del Sistema

Diagrama de Arquitectura del Prototipo v0

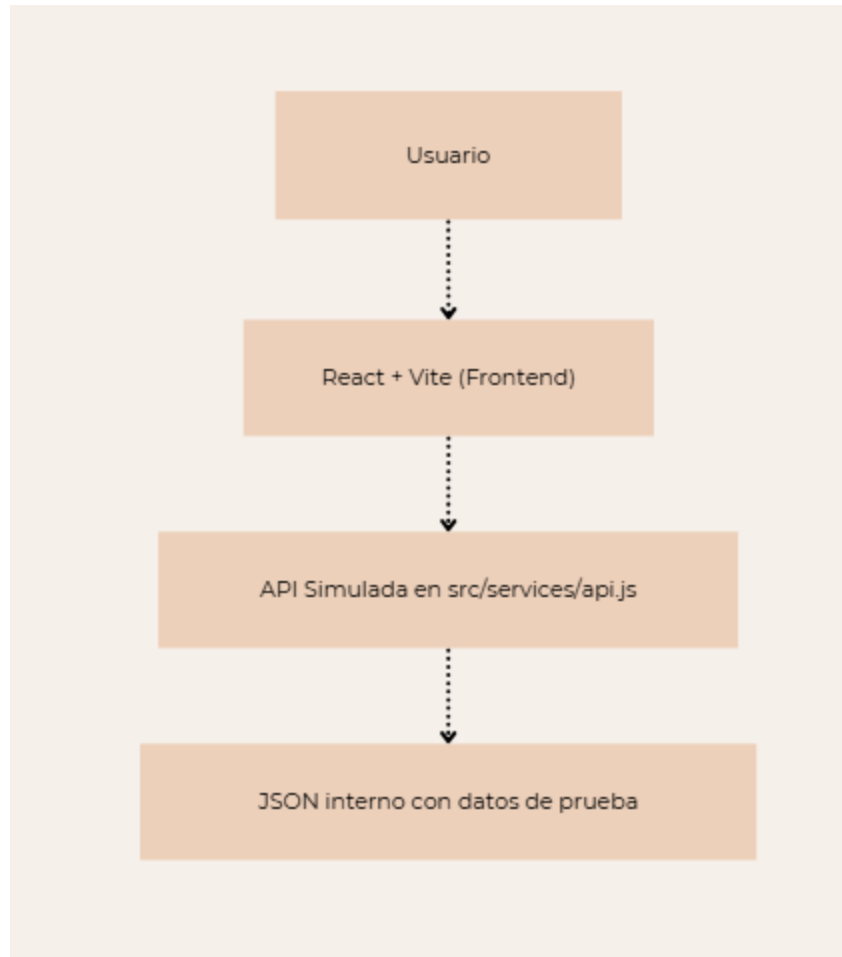
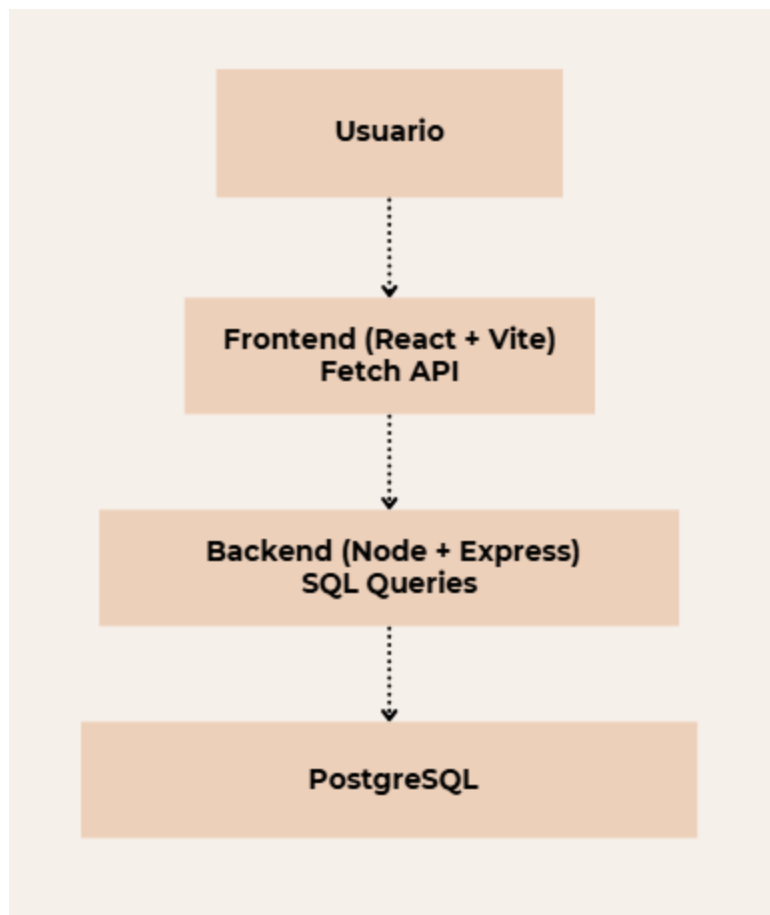


Diagrama de Arquitectura del Sistema Final



Planificación y Costos

TAREA	HORAS
ANÁLISIS Y DEFINICIÓN DEL PROBLEMA	3
DISEÑO UI + ARQUITECTURA	4
IMPLEMENTACIÓN DE API SIMULADA	3
IMPLEMENTACIÓN DEL FRONTEND (REACT + VITE)	10
LÓGICA DE FILTROS Y RECOMENDACIONES	5
ESTILOS E INTERFAZ FINAL	5
PRUEBAS Y AJUSTES	4
DOCUMENTACIÓN TÉCNICA + PRESENTACIÓN	5

Total estimado: 39 horas

Con 20% de colchón: $46.8 \approx 47$ horas

Presupuesto:

Costo por hora del desarrollador: **Q50.00**

$47 \text{ horas} * Q50.00 = Q2,350.00$

En conclusión, GameMind demuestra que es posible crear experiencias personalizadas basadas en el estado emocional del usuario y sus intereses educativos. El prototipo v0 construido en **React + Vite** es totalmente funcional y sirve como base para un sistema más avanzado con backend real.